

AegisRx

Pharmaceutical Supply Chain & Patient Safety Ecosystem

Software Design Analysis (SDA)

Student Name	Student ID	Role
Turab Qasim	24I-0600	Team Manager + QA Manager
Abdullah Tanzeem	24I-0742	Analyst + Developer
Wasif Mehmood Mughal	24I-0699	Designer + Developer

Submitted to: Miss Afia Zafar

Table of Contents

1. Introduction.....	4
1.1 Purpose.....	4
1.2 Product Scope.....	4
1.3 Title	4
1.4 Objectives	5
1.5 Problem Statement.....	5
2. Overall Description	5
2.1 Product Perspective	6
2.2 Product Functions.....	6
2.3 List of Use Cases	7
2.4 Extended Use Cases	8
2.4.1 UC-02 Extension — Partial Batch Transfer	8
2.4.2 UC-05 Extension — Adaptive Interaction Strategy.....	9
2.5 Use Case Diagram	9
3. Nonfunctional Requirements.....	11
3.1 Performance Requirements.....	11
3.2 Safety Requirements	11
3.3 Security Requirements	12
3.4 Software Quality Attributes	12
3.5 Business Rules.....	13
3.6 Operating Environment.....	13
3.7 User Interfaces	14
4. Domain Model	15
4.1 Entity Descriptions.....	16
4.2 Key Relationships.....	16
5. System Sequence Diagram	18
5.1 Scenario: Patient Locates Medicine via Smart Locator	18
6. Sequence Diagram	19
6.1 Scenario: Manufacturer Initiates a Partial Batch Transfer	19
7. Class Diagram	21
7.1 Service Layer Classes	21
7.2 Design Pattern Summary.....	23
8. Package Diagram	23
8.1 Package Descriptions.....	23
8.2 Package Dependency Matrix.....	24

9. Deployment Diagram 24

9.1 Deployment Nodes 24

9.2 Communication Protocols 25

9.3 Deployment Considerations..... 25

10. Glossary of Terms 25

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) provides a complete and formal description of the AegisRx system, covering its functional requirements, nonfunctional constraints, system architecture, and design artefacts. This document is intended for a diverse audience including project managers, software developers, quality assurance engineers, database administrators, and stakeholders involved in decision-making.

The document defines the expected behaviour of the system in sufficient detail to allow development teams to design, implement, and test the solution without ambiguity. It also serves as the contractual baseline against which the delivered system will be evaluated.

1.2 Product Scope

AegisRx is a comprehensive, multi-role desktop application targeting the pharmaceutical supply chain and patient medication safety domain. It operates at the intersection of three user communities:

- Medicine Manufacturers who need to register, track, and transfer pharmaceutical batches.
- Pharmacies that receive batch transfers, manage verified inventory, and serve patients.
- Patients who require a trusted mechanism to verify medicine authenticity, locate verified stock, manage their medication profiles, and receive AI-driven dietary and medical recommendations.

The system introduces a Global Batch Identifier (GBI) — a unique, tamper-evident token assigned to every medicine batch — which enables end-to-end traceability through the supply chain. This GBI is the fundamental building block that connects all stakeholder interactions.

1.3 Title

AegisRx

Pharmaceutical Supply Chain and Patient Safety Ecosystem



1.4 Objectives

The primary objectives of AegisRx are:

- Combat counterfeit medicines by assigning tamper-evident Global Batch Identifiers (GBIs) to every registered medicine batch, enabling full chain-of-custody traceability from manufacturer to pharmacy.
- Provide a Smart Locator service that allows patients to search for verified, in-stock medicines within a chosen geographic region, reducing dependence on informal supply channels.
- Enforce patient safety by automatically analysing all active medications in a patient profile for potential drug-to-drug interactions each time a new medication is added.
- Monitor dietary inputs against active medications in real time to detect and alert patients about dangerous food-drug conflicts before harm occurs.
- Leverage Google Gemini AI to deliver personalised, context-aware dietary recommendations and to answer patient queries based on their individual medication history.
- Implement role-based access control (RBAC) to ensure that sensitive operations are restricted to the appropriate actors, protecting data integrity and patient privacy.

1.5 Problem Statement

The global pharmaceutical sector faces two intertwined crises: the proliferation of counterfeit medicines and the rising incidence of preventable adverse drug reactions (ADRs). According to the World Health Organization, counterfeit medicines constitute a significant proportion of the pharmaceutical market in developing economies, resulting in treatment failure, drug resistance, and loss of life. Simultaneously, patients managing multiple chronic conditions frequently carry two or more active prescriptions, making them vulnerable to harmful drug-to-drug interactions and fooddrug conflicts that are rarely communicated clearly at the point of care.

The existing supply chain for pharmaceuticals lacks the transparency necessary to allow a patient or pharmacist to independently verify the origin and authenticity of a medicine batch. Without a standardised, tamper-evident tracking mechanism, counterfeit products can enter the supply chain at any point and become indistinguishable from legitimate stock.

AegisRx addresses these challenges by enforcing a transparent chain of custody through Global Batch Identifiers, automating drug interaction screening at the point of medication entry, and harnessing the capabilities of generative AI to provide patients with accessible, real-time medical guidance — bringing pharmaceutical-grade safety features directly to the end user's desktop.

2. Overall Description

2.1 Product Perspective

AegisRx is designed as a self-contained, multi-tiered desktop application built on the Java platform. It does not depend on any pre-existing enterprise resource planning system, but it is designed to co-exist with typical pharmacy management software by exposing a clearly defined data layer. The architecture follows a classical three-tier model:

Layer	Technology & Responsibility
Presentation	JavaFX 21 — role-specific dashboards with reactive UI components. Handles all user input validation and visual feedback.
Business Logic	Service classes (BatchService, AIService, DrugInteractionService, SmartLocatorService) implementing domain rules, design patterns, and orchestration logic.
Data Access	DAO classes communicating with Microsoft SQL Server over JDBC. Manages transactional integrity during batch transfers and medication updates.
External Integration	RESTful HTTPS calls to Google Gemini AI API using JSON payloads serialised via Gson. Supports graceful fallback to local logic when the API is unavailable.

2.2 Product Functions

The following table provides a structured summary of the major functional capabilities of AegisRx:

Function	Description	Primary Actor
Batch Management	Register new medicine batches, transfer full or partial quantities, and verify GBI authenticity.	Manufacturer / Pharmacy
Smart Locator	Search for verified, in-stock medicines across registered pharmacies in a selected region.	Patient
Drug Interaction Check	Analyse all active patient medications for known or AI-detected drug-to-drug interaction risks.	Patient / System
Dietary Monitoring	Log daily food entries and cross-reference them against active medications to identify dietary conflicts.	Patient
AI Smart Suggestions	Submit natural-language queries to Gemini AI; receive personalised medical and dietary guidance contextualised to the patient profile.	Patient

Role-Based Access Control	Enforce distinct dashboard views and operation permissions for Admin, Manufacturer, Pharmacy Staff, and Patient roles.	System / Admin
----------------------------------	--	----------------

2.3 List of Use Cases

UC ID	Use Case Name	Primary Actor	Brief Description
UC-01	Register Medicine Batch	Manufacturer	Create a new batch with GBI, production/expiry dates, and initial quantity.
UC-02	Transfer Medicine Batch	Manufacturer	Transfer full or partial batch to a target pharmacy; auto-split on partial transfers.
UC-03	Check Drug Interactions	Patient	Automatically evaluate all active patient medications for interaction risks.
UC-04	Locate Verified Medicine	Patient	Search for a specific drug by name and region to find verified pharmacies with stock.
UC-05	Verify Medicine Authenticity	Pharmacy	Scan or enter a GBI to confirm a batch is legitimate and trace its chain of custody.
UC-06	Log Food Intake	Patient	Record a dietary intake event for subsequent dietary conflict analysis.
UC-07	Receive Dietary Conflicts Alert	Patient	Identify food items that conflict with the patient's active medication regimen.

UC-08	Register Medication Profile	Patient	Add a verified medication to the patient's active profile with dosage and frequency; enables the Safety Logic Engine for continuous monitoring.
UC-09	Purchase Medicine	Patient/Pharmacy	Process a point-of-sale transaction; system performs an automated interaction check, deducts stock from inventory, and updates the patient's profile.
UC-10	Mark Medicine as Destroyed	Pharmacy	Formally remove expired, damaged, or recalled medicine from inventory and permanently update the global batch status to 'Destroyed'.
UC-11	Update Pharmacy Inventory	Pharmacy	Digitally receive incoming shipments from manufacturers, verify the recipient ID, and update batch status to 'In Stock' to reflect current availability.
UC-12	Manage User Accounts	Admin	Create user profiles for Manufacturers, Pharmacies, and Patients; ensures secure, role-based access control across the system.

2.4 Extended Use Cases

2.4.1 UC-02 Extension — Partial Batch Transfer

When a manufacturer initiates a batch transfer where the transferred quantity is less than the current batch quantity, the system executes an automatic batch-split operation:

- The original batch record is updated to reflect the remaining quantity.
- A new sub-batch record is created with the transferred quantity and is assigned a derived sub-GBI that encodes the parentage of the original batch.

- The sub-GBI maintains referential integrity with the parent GBI, ensuring the entire provenance chain can be reconstructed at audit time.
- Both the parent and sub-batch records are persisted atomically within a single database transaction.

2.4.2 UC-05 Extension — Adaptive Interaction Strategy

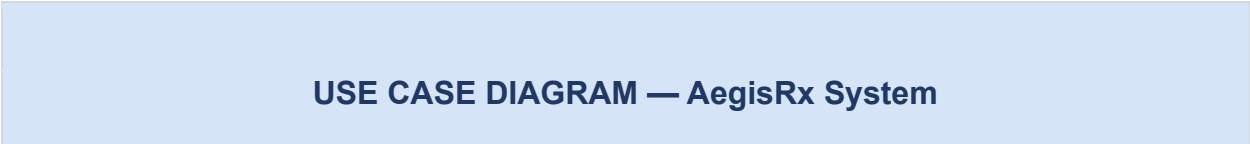
The Drug Interaction checking subsystem implements the Strategy design pattern to allow dynamic switching between two implementations:

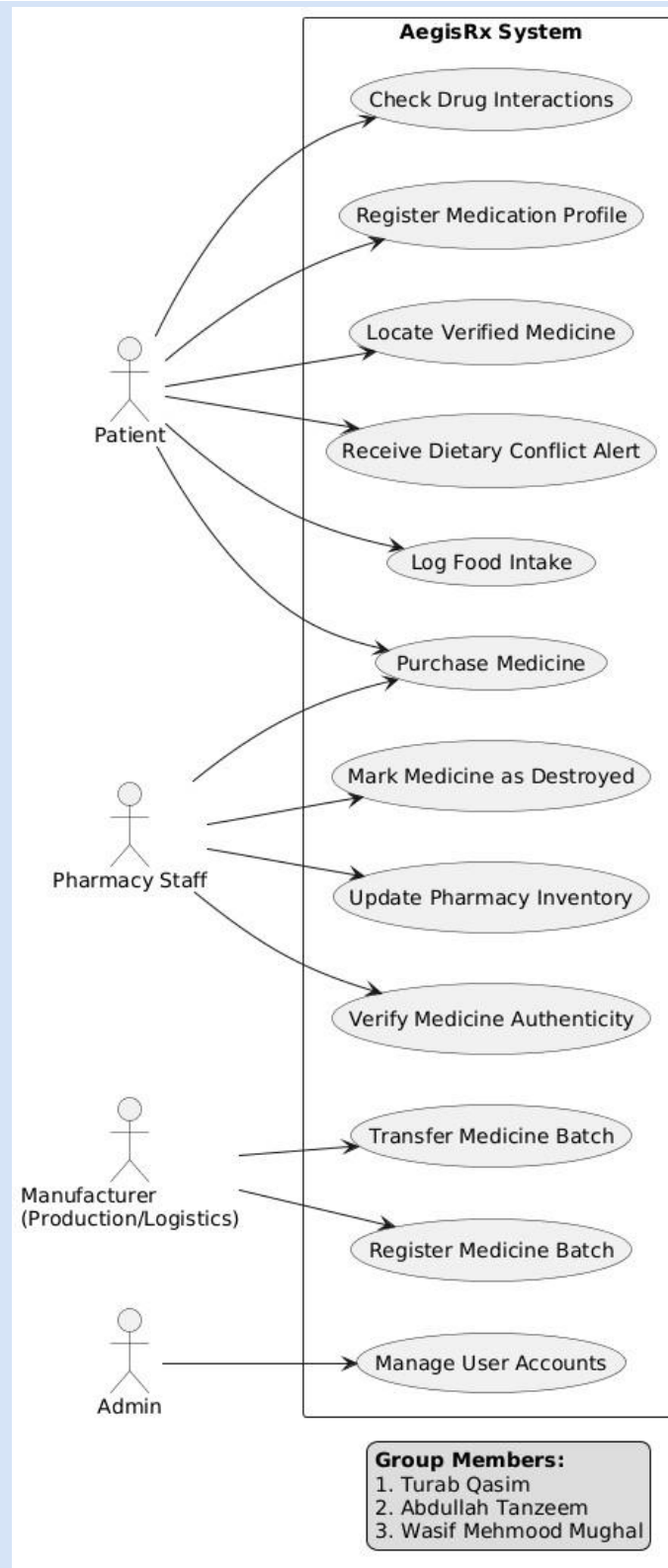
Strategy	Behaviour
AI Interaction Checker	Sends the patient medication list to the Gemini AI API via GeminiClient and parses the response for interaction warnings. Used when the API is reachable and configured.
Local Interaction Checker	Uses a locally maintained drug interaction rule database to perform checks. Activated automatically when the Gemini API is unavailable, ensuring uninterrupted safety coverage.

The DrugInteractionService exposes a setStrategy() method that allows the application to switch strategies at runtime based on API health checks performed during startup or on demand.

2.5 Use Case Diagram

The following diagram illustrates the relationships between system actors and primary use cases:





3. Nonfunctional Requirements

3.1 Performance Requirements

ID	Requirement	Rationale
PR-01	Local drug interaction checks must complete within 500 milliseconds.	Ensures a responsive UI experience for patients adding medications.
PR-02	The Gemini AI API call must enforce a 10second timeout to prevent UI blocking.	Prevents indefinite hangs; allows graceful fallback to local checking.
PR-03	Database queries for the Smart Locator must execute in under 1 second.	Patients searching for medicine need immediate feedback to act quickly.
PR-04	The application startup time must not exceed 5 seconds on a standard desktop machine.	Minimises user frustration during session initialisation.

3.2 Safety Requirements

ID	Requirement	Mitigation Strategy
SR-01	The system must validate that the expiry date of every batch is strictly greater than its production date at registration time.	Server-side validation prevents logically invalid batches from being persisted.
SR-02	If the Gemini AI API is unreachable, the system must fall back to local rule-based interaction checking without user intervention.	Guarantees that safety checks are never silently skipped due to connectivity failures.
SR-03	The system must display a clear, prominent warning when a drug-to-drug interaction or dietary conflict of HIGH severity is detected.	High-visibility alerts ensure patients do not dismiss critical safety information.
SR-04	Medicines with a status other than ACTIVE must be prevented from being transferred or dispensed.	Prevents distribution of recalled, expired, or quarantined batches.

3.3 Security Requirements

ID	Requirement	Implementation Note
SEC-01	All user passwords must be hashed using a strong one-way hashing algorithm (e.g., BCrypt) before storage.	Plaintext passwords must never be written to the database or to log files.
SEC-02	The system must enforce role-based access control: Manufacturers may only access batch management; Patients may only access their own profiles.	Enforced at the service layer; UI controllers verify the authenticated role on load.
SEC-03	All API communications with the Gemini AI endpoint must use HTTPS with TLS 1.2 or higher.	Prevents interception of sensitive medication profile data transmitted in prompts.
SEC-04	Patient medication and dietary data must not be accessible to users in the Manufacturer or Pharmacy Staff roles.	Protects personally identifiable health information in compliance with data privacy norms.

3.4 Software Quality Attributes

Attribute	Description	Implementation Mechanism
Maintainability	The codebase uses established design patterns (Strategy, Factory, Observer, Pure Fabrication) to keep components loosely coupled and independently replaceable.	Clearly defined interfaces between the UI, service, and DAO layers.
Reliability	Critical data mutations (batch transfers, medication updates) are wrapped in database transactions to ensure atomicity and prevent partial writes.	SQL Server transaction management via JDBC; rollback on any exception.
Interoperability	The application integrates with an external RESTful API over HTTPS using JSON, ensuring compatibility with industry-standard web services.	Gson library for JSON serialisation/deserialisation of Gemini API payloads.

Usability	Role-specific dashboards minimise cognitive load by presenting only the operations relevant to the currently authenticated user.	JavaFX FXML layouts with intuitive navigation and descriptive labels.
Testability	The Strategy pattern for interaction checking allows the AI-based strategy to be swapped with a mock strategy during unit testing without modifying production code.	Dependency injection via <code>setStrategy()</code> method on <code>DrugInteractionService</code> .

3.5 Business Rules

BR ID	Rule	Enforcement Point
BR-01	Only the owning manufacturer may initiate a batch transfer.	<code>BatchService.transferBatch()</code> validates <code>manufacturerId</code> against the batch owner.
BR-02	Medicine batches with status other than ACTIVE cannot be transferred.	Status check performed in <code>BatchService</code> before transfer is processed.
BR-03	A patient must have at least two active medications before the interaction checker runs.	<code>DrugInteractionService.checkPatientInteractions()</code> returns immediately if medication count < 2.
BR-04	A batch expiry date must be strictly greater than its production date.	Enforced via server-side validation in <code>BatchService.registerBatch()</code> .
BR-05	Transferred quantity cannot exceed the current available quantity of the batch.	Quantity validation check in <code>BatchService</code> prior to split-or-full-transfer logic.

3.6 Operating Environment

Component	Specification
Runtime Platform	Java Virtual Machine (JVM), version 17 or higher (LTS release recommended).

UI Framework	JavaFX 21. Requires the JavaFX runtime to be available on the host machine or bundled with the application distribution.
Operating System	Windows 10/11, macOS 12+, or a modern Linux distribution with a graphical desktop environment.
Database	Microsoft SQL Server (2019 or later). Accessed via a JDBC connection. A valid SQL Server instance with the AegisRx schema must be reachable from the client machine.
External API	Google Gemini AI API. Requires an active internet connection and a valid API key configured in the application properties file.
Minimum Hardware	Dual-core processor, 4 GB RAM, 500 MB available disk space for the application and its dependencies.

3.7 User Interfaces

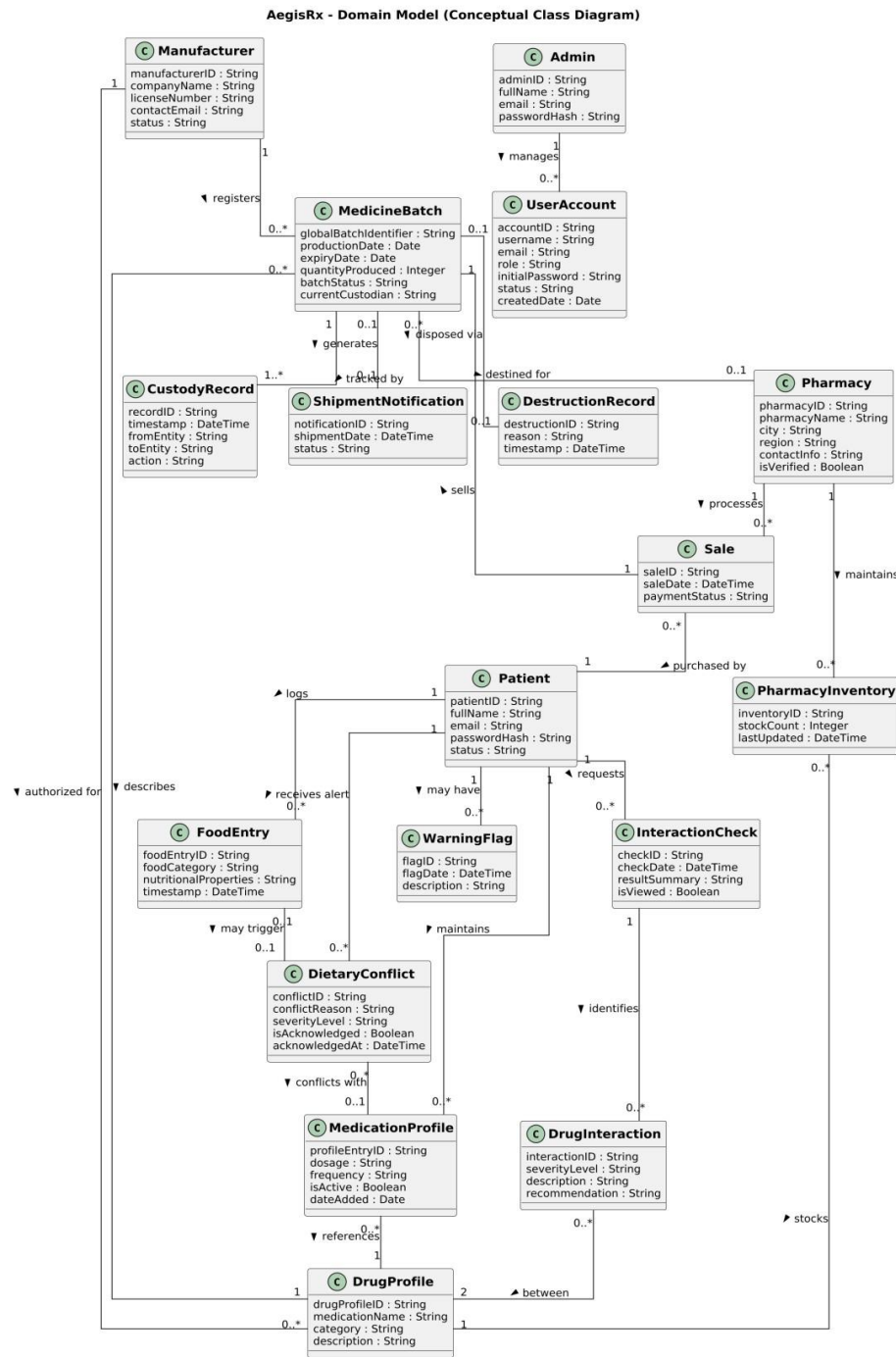
AegisRx delivers a rich, role-specific graphical user interface constructed with JavaFX 21 FXML. Each dashboard is tailored to the responsibilities of the authenticated role:

Dashboard	Key UI Components & Features
Admin Dashboard	User management panel, system health monitoring, role assignment controls, audit log viewer.
Manufacturer Dashboard	Batch registration form (drug name, GBI auto-generation, date pickers, quantity input), batch transfer form, supply chain timeline view, batch status filter.
Pharmacy Dashboard	Incoming batch reception panel, current inventory table, GBI verification scanner, point-of-sale interface.
Patient Dashboard	Active medication list, Add Medication button (triggers interaction check), dietary log form, Smart Locator search bar, AI chat panel with message history and prompt input.

All dashboards share a consistent navigation header with the AegisRx branding, the authenticated user's name and role badge, and a Logout button. Form validation errors are displayed inline adjacent to the offending field using a standardised red error label style.

4. Domain Model

The domain model captures the primary entities within the AegisRx problem space and their relationships. The model is intentionally technology-agnostic and represents real-world concepts rather than implementation details.



4.1 Entity Descriptions

Entity	Key Attributes	Description
User	userId, username, role, status	Abstract base entity representing any authenticated principal in the system. Specialised by Manufacturer, Patient, and PharmacyStaff.
Manufacturer	companyName (+ inherited)	A pharmaceutical company registered in the system. Has exclusive rights to register and transfer medicine batches.
Patient	conditions, allergies (+ inherited)	An end user who maintains a medication profile, logs dietary intake, and receives safety alerts and AI recommendations.
PharmacyStaff	pharmacyId (+ inherited)	A staff member associated with a specific pharmacy. Responsible for receiving batch transfers and verifying GBI authenticity.
Medication	medId, drugName, dosage, frequency, active	Represents a drug prescribed to a patient. The active flag determines whether the medication participates in interaction checks.
MedicineBatch	gbi, productionDate, expiryDate, quantity, status	A traceable, uniquely identified batch of a physical medicine. Travels from manufacturer through to pharmacy. Status lifecycle: ACTIVE -> TRANSFERRED -> EXPIRED.
DietaryAlert	alertId, severity, description	Generated by the system when a logged food entry conflicts with an active patient medication. Severity levels: LOW, MEDIUM, HIGH.

4.2 Key Relationships

Relationship	Multiplicity	Semantics
Patient takes Medication	1 to *	A patient may have one or more medications in their profile. Each medication belongs to exactly one patient.
Patient receives DietaryAlert	1 to *	Each dietary log event may generate zero or more alerts depending on food-drug conflict analysis results.

Manufacturer produces MedicineBatch	1 to *	A manufacturer registers one or more batches. Each batch has exactly one owning manufacturer.
User specialises into roles	1 to 1	Inheritance: every Manufacturer, Patient, and PharmacyStaff IS-A User with an augmented attribute set.

5. System Sequence Diagram

This section presents the System Sequence Diagram (SSD) for the primary patient-facing use case: locating a medicine through the Smart Locator service. The SSD captures the externally visible messages exchanged between the Patient actor and the AegisRx system as a black box.

5.1 Scenario: Patient Locates Medicine via Smart Locator

Step	Actor / System Action	System Response / Output
1	Patient enters drug name and selects a target region; submits the Smart Locator form.	System validates that a drug was selected and a region was specified.
2	System calls <code>SmartLocatorService.locateMedicine(drugId, region)</code> .	Service forwards query to the database via DAO: <code>findWithDrugInStock(drugId, region)</code> .
3	Database executes the supply chain query and returns a list of pharmacies with verified stock.	Results returned to the service layer as a typed list.
4a	[If pharmacies found] Service returns nonempty list to the UI controller.	Patient Dashboard renders a table listing pharmacy names, locations, and available quantity.
4b	[If no pharmacies found] Service returns an empty list.	UI displays the message: 'No verified stock found in the selected region.' and suggests broadening the search area.

Note

The SSD treats the AegisRx system as a black box. Internal component interactions are detailed in the full Sequence Diagram in Section 6.

6. Sequence Diagram

This section provides a detailed internal sequence diagram for the batch transfer scenario, illustrating how control flows through the service and DAO layers during a partial batch transfer operation.

6.1 Scenario: Manufacturer Initiates a Partial Batch Transfer

#	Caller	Callee / Message	Description
1	Manufacturer (UI)	BatchService.transferBatch(batchId, manId, pharmId, qty)	User submits the Transfer Batch form with target pharmacy and quantity.
2	BatchService	BatchDAO.findById(batchId)	Service retrieves the full batch record to validate ownership and status.
3	BatchDAO	return MedicineBatch	DAO returns the hydrated batch entity.
4	BatchService	batch.getQuantity()	Service reads current quantity. Determines transfer is partial (qty < totalQty).
5	BatchService	batch.setQuantity(total - qty)	Parent batch quantity reduced by the transferred amount.
6	BatchService	BatchDAO.update(batch)	Persists the updated parent batch to the database.
7	BatchService	create subBatch (qty, derived subGBI)	A new MedicineBatch object is instantiated with the transferred quantity and a sub-GBI encoding parent lineage.
8	BatchService	BatchDAO.insert(subBatch)	Sub-batch is persisted; DAO returns the new subbatch ID.

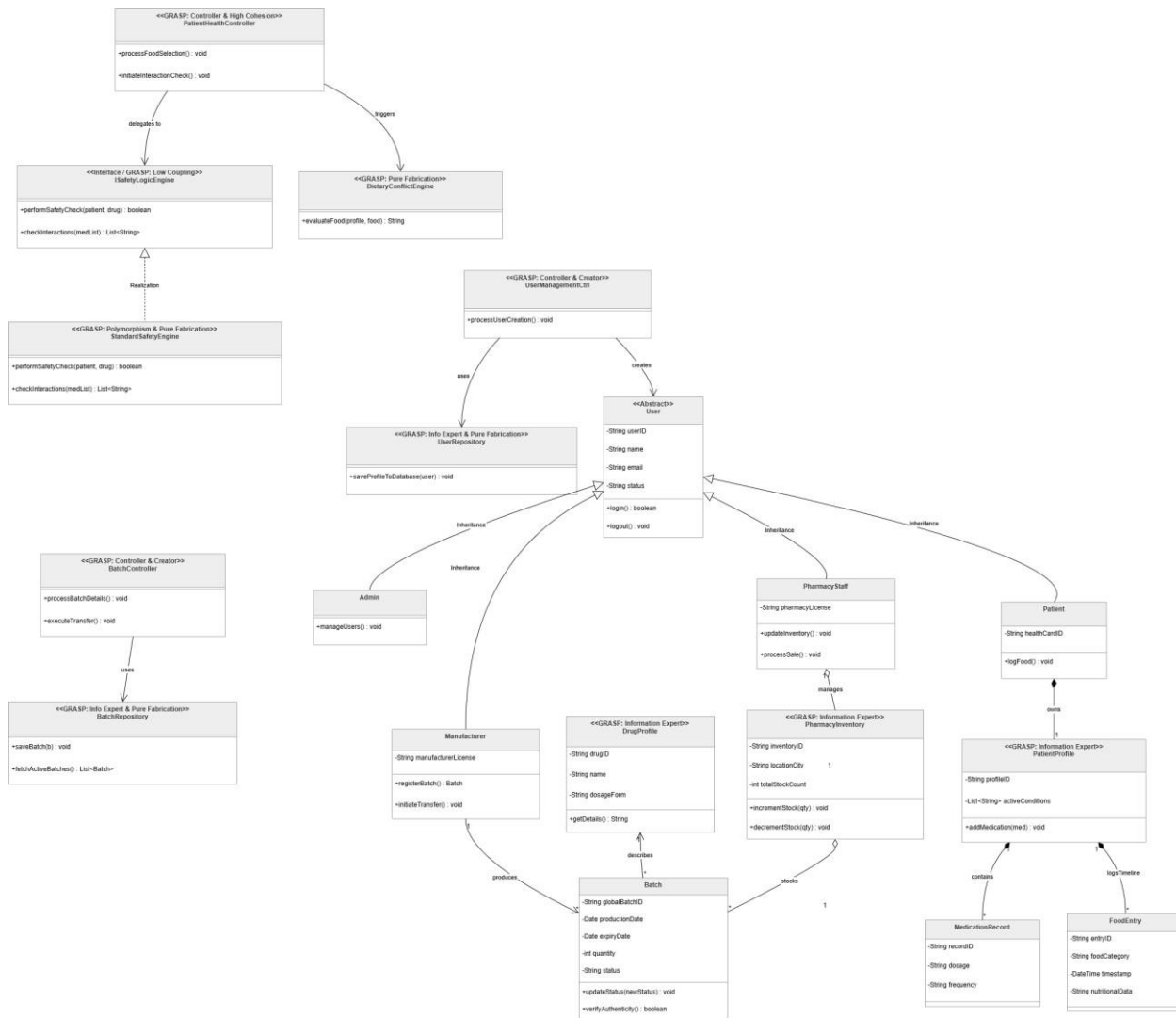
9	BatchService	BatchDAO.insertTransfer(subBatchId, manId, pharmId)	Transfer record linking the sub-batch to the source manufacturer and target pharmacy is created.
10	BatchService	return success to Manufacturer UI	UI displays confirmation: sub-GBI, transferred quantity, and target pharmacy name.

**Design
Note**

Steps 5 through 9 are executed within a single JDBC transaction. If any step fails, the transaction is rolled back to preserve data integrity.

7. Class Diagram

This section describes the service layer class structure with a focus on the AI integration subsystem. The design demonstrates the Strategy and Dependency Inversion principles applied to the interaction checking functionality.



7.1 Service Layer Classes

Class / Interface	Key Methods / Fields	Responsibilities
-------------------	----------------------	------------------

BatchService	registerBatch(), transferBatch(), verifyAuthenticity()	Manages the full lifecycle of medicine batches including GBI generation, partial transfers, and authenticity verification.
SmartLocatorService	locateMedicine(drugId, region), searchDrugs(name)	Executes regional pharmacy stock queries and drug profile searches on behalf of the patient.
AIService	askQuestion(patientId, question), getDietaryRecommendation(patientId)	Constructs context-aware prompts from the patient medication profile and routes them to GeminiClient.
DrugInteractionService	checkPatientInteractions(patientId), setStrategy(strategy)	Orchestrates interaction checking by delegating to the currently configured InteractionCheckStrategy.
«interface» InteractionCheckStrategy	checkInteractions(meds), getStrategyName()	Defines the contract for pluggable interactionchecking algorithms. Enables the Strategy pattern.
AllInteractionChecker	checkInteractions(meds) — via Gemini API	Implements InteractionCheckStrategy using Google Gemini AI for richer, AI-augmented interaction detection.
LocalInteractionChecker	checkInteractions(meds) — via local rule DB	Implements InteractionCheckStrategy using a local drug interaction knowledge base. Fallback when API is unavailable.
GeminiClient	generateContent(prompt)	Low-level HTTP client that constructs JSON payloads, calls the Gemini REST endpoint, and parses text responses. Used by both AIService and AllInteractionChecker.

7.2 Design Pattern Summary

Pattern	Applied To	Benefit
Strategy	DrugInteractionService	Allows runtime switching between AI and local interaction checking without modifying the service interface.
Factory	DAO / Service creation	Centralises instantiation logic; allows dependency injection and easier unit testing.
Observer	DietaryAlert generation	Decouples the food logging event from the alert generation logic; allows multiple listeners without tight coupling.
Pure Fabrication	GeminiClient	GeminiClient does not map to a domain concept; it is a fabricated class that encapsulates external API complexity.

8. Package Diagram

The package diagram illustrates the high-level logical grouping of classes within the AegisRx codebase and the dependency relationships between packages. The architecture enforces a strict downward dependency flow.

8.1 Package Descriptions

Package	Contents & Responsibilities
com.aegisrx.domain	Plain Java objects (POJOs) representing domain entities: User, Medication, MedicineBatch, DietaryAlert, and their subclasses. No business logic or database code.
com.aegisrx.dao	Data Access Objects (DAOs) responsible for all SQL queries and database mutations. Contains UserDAO, BatchDAO, and the DatabaseConnectionManager singleton.

com.aegisrx.service	Core business logic layer. Contains BatchService, AIService, SmartLocatorService, and DrugInteractionService. Depends on DAO and domain packages.
com.aegisrx.ai	AI integration utilities including GeminiClient (HTTP communication) and PromptTemplates (prompt construction helpers). Isolated to allow independent testing.
com.aegisrx.ui.controllers	JavaFX FXML controllers for each dashboard view. Depends only on the service layer; never accesses DAOs or the AI package directly.

8.2 Package Dependency Matrix

Package (depends on →)	domain	dao	service	ai	ui
com.aegisrx.domain	—	No	No	No	No
com.aegisrx.dao	Yes	—	No	No	No
com.aegisrx.service	Yes	Yes	—	Yes	No
com.aegisrx.ai	No	No	No	—	No
com.aegisrx.ui.controllers	No	No	Yes	No	—

Layering Principle	Dependencies only flow downward (UI -> service -> dao -> domain). The ai package is a lateral utility used exclusively by the service layer. This strict layering prevents circular dependencies and improves maintainability.
--------------------	--

9. Deployment Diagram

The deployment diagram illustrates the physical nodes on which the AegisRx system components execute and the communication protocols used between them.

9.1 Deployment Nodes

Node	Hosted Component	Description
------	------------------	-------------

Client Device (Desktop/Laptop)	AegisRx JavaFX Application within JRE 17+	The primary user-facing node. All four role dashboards and the AI chat panel run on this machine. No server-side rendering is required.
Database Server	Microsoft SQL Server + AegisRx DB	Centralised persistence node. Stores all user, batch, medication, and dietary data. May be hosted on a dedicated server or a cloudmanaged SQL Server instance.
Cloud Services (Google)	Google Gemini AI API	External SaaS node providing generative AI capabilities. The client device communicates with this endpoint over HTTPS. No proprietary server infrastructure is required from the AegisRx team.

9.2 Communication Protocols

Source Node	Target Node	Protocol & Details
Client Device	Database Server	JDBC over TCP/IP. Connection string configured in the application properties file. Supports both Windows Authentication and SQL Server Authentication.
Client Device	Google Cloud (Gemini API)	HTTPS / TLS 1.2+. JSON request/response bodies. API key transmitted via HTTP header. 10-second timeout enforced at the client.

9.3 Deployment Considerations

- The AegisRx application may be distributed as a self-contained JAR or as a platformspecific installer that bundles the JRE 17 runtime to eliminate dependency on a pre-installed Java environment.
- The SQL Server instance should be placed within the same local network as the client machines or behind a VPN to minimise query latency and prevent unauthorised external access.
- The Gemini API key must be stored in a configuration file that is excluded from version control. For production deployments, consider using environment variables or a secrets management solution.

- All database communication should be encrypted via SQL Server's built-in TLS channel encryption to protect sensitive health data in transit.

10. Glossary of Terms

Term / Acronym	Definition
ADR	Adverse Drug Reaction — an unintended and harmful response to a medication taken at a normal therapeutic dose.
GBI	Global Batch Identifier — a unique, tamper-evident token generated by AegisRx upon batch registration, used to trace the provenance of a medicine batch.
Gemini AI	Google DeepMind Gemini — the large language model API used by AegisRx to provide AI-powered drug interaction analysis and dietary recommendations.
Gson	Google's open-source Java library used to serialise and deserialise JSON objects for communication with the Gemini REST API.
HTTPS	Hypertext Transfer Protocol Secure — the encrypted communication protocol used for all API calls to external services.
JDBC	Java Database Connectivity — the standard Java API for connecting to relational databases, used to interface with Microsoft SQL Server.
JavaFX	An open-source client application platform for Java that provides a modern framework for building rich desktop graphical user interfaces.
JRE / JVM	Java Runtime Environment / Java Virtual Machine — the execution environment required to run the AegisRx application.
RBAC	Role-Based Access Control — a security model that restricts system access based on the authenticated user's assigned role.
SRS	Software Requirements Specification — this document, providing a complete description of AegisRx system requirements.
Strategy Pattern	A behavioural design pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime.
Sub-GBI	A derived Global Batch Identifier generated during a partial batch transfer to uniquely identify the split sub-batch while preserving its lineage to the parent batch.